Three address Code

* Three address code is a sequence of statements of the form,

  $x = y \text{ op } z$, where x, y and z are names, constants or compiler generated temporaries.

* Consider, $x + y * z$, might be translated in to a sequence,

$$t1 = y * z$$
$$t2 = x + t1$$

where $t1$ and $t2$ are compiler generated temporary names.

* Consider the expression $a = b * -c + b * -c$.

$$t1 = -c$$
$$t2 = b * t1$$
$$t3 = -c$$
$$t4 = b * t3$$
$$t5 = t2 + t4$$
$$a = t5$$

Fig (a)

$$t1 = -c$$
$$t2 = b * t1$$
$$t5 = t2 + t2$$
$$a = t5$$

Fig (b)

Fig. Code for the syntax tree fig (a) and DAG fig (b)

* The reason for the term 'three address code' is that each statement usually contains three addresses, two for the operands and one for the result.

Types of three address statements

* Statements of the form,

$$x = y \text{ op } z$$
$$x = \text{op } y$$
$$x = y$$
$$\text{if } x \text{ relop } y \text{ goto } L$$

⋮

Implementation of Three adress statements

* A three address statement is an abstract form of intermediate code. These statements can be implemented as records with fields for the operator and the operands. Three such representations are quadruples, triples and indirect triples

Quadruples

* A quadruple is a record structure with four fields which we call op, arg1, arg2 and result.

The quadruple for the expression $a = b * -c + b * -c$ is given as follows

Quadruples

	Op	arg1	arg2	Result
(0)	uminus	c	–	t1
(1)	*	b	t1	t2
(2)	uminus	c		t3
(3)	*	b	t3	t4
(4)	+	t2	t4	t5
(5)	=	t5		a

Triples

* To avoid entering temporary names into the symbol table, refer to a temporary value by the position of the statement that compute it.
* Then the three-address statements can be represented by records with only three fields, op, arg1 and arg2.

	OP	arg1	arg2
(0)	uminus	c	
(1)	*	b	(0)
(2)	uminus	c	
(3)	*	b	(2)
(4)	+	(1)	(3)
(5)	assign	a	(4)

* The fields arg1 and arg2, for the arguments of op are either pointers to the symbol table ~~for~~ or pointers into the triple structure.

Indirect triples

* Another implementation of three-address code that has been considered is that of listing pointers to triples. This is called indirect triples.

	statement
(0)	(14)
(1)	(15)
(2)	(16)
(3)	(17)
(4)	(18)
(5)	(19)

	op	arg1	arg2
(14)	uminus	c	
(15)	*	b	(14)
(16)	uminus	c	
(17)	*	b	(16)
(18)	+	(15)	(17)
(19)	assign	a	(18)

Fig. Indirect triples representation of three-address statement.

Comparison of Representation: ~~The~~

* Quadruples appears in an optimizing compiler where statements are often moved around. However in triples moving a statement that defines a temporary value requires us to change all references to that statement in the arg1 and arg2 arrays. This problem

makes difficult to use in an optimizing compiler. Indirect triples present no such problem. A statement can be moved by reordering the statement list. Since pointers to temporary values refer to op-arg1-arg2 arrays, which are not changed, none of these pointers need be changed. Thus indirect triples look very much like quadruples as far as their utility is concerned.

* The two notations require about the same amount of space and they are equally efficient for reordering of code.
* Storage requirement is less in triples compared to quadruples and indirect triples.

Code Optimization

* The code produced by compiler can be made to run faster or take less space or both. This is achieved by program transformations called optimizing compiler.
* Program transformations that improve the target code without taking into consideration any properties of the target machine is called machine independent optimizations

Criteria for Code Improving Transformation.

(1) A transformation must preserve the meaning of programs or it must not change the output produced by a program for the given input.

(2) The transformation must on the average, speed up programs by a measurable amount.

(3) Reducing the space taken by the compiled code.

(4) A transformation must be worth the effort.

The Principal sources of Optimization.

* A transformation of a program is called local if it can be performed by looking only at the statements in a basic block, otherwise it is called global. Local transformations are usually performed first.

Function preserving Transformations

* There are a number of ways in which a compiler can improve a program without changing the function it computes.

* Common subexpression elimination, copy propagation, dead code elimination and constant folding are common examples of function preserving transformati

Common subexpression elimination

* An occurrence of an expression E is called a common subexpression if E was previously computed, and the values of variables in E have not changed since the previous computation.

Consider

B5

```
t6 = 4*i
X = a[t6]
t7 = 4*i
t8 = 4*j
t9 = a[t8]
a[t7] = t9
t10 = 4*j
a[t10] = X
goto B2
```

(a) Before

B5

```
t6 = 4*i
X = a[t6]
t8 = 4*j
t9 = a[t8]
a[t6] = t9
a[t8] = X
goto B2
```

(b) After

Fig. Local common subexpression elimination.

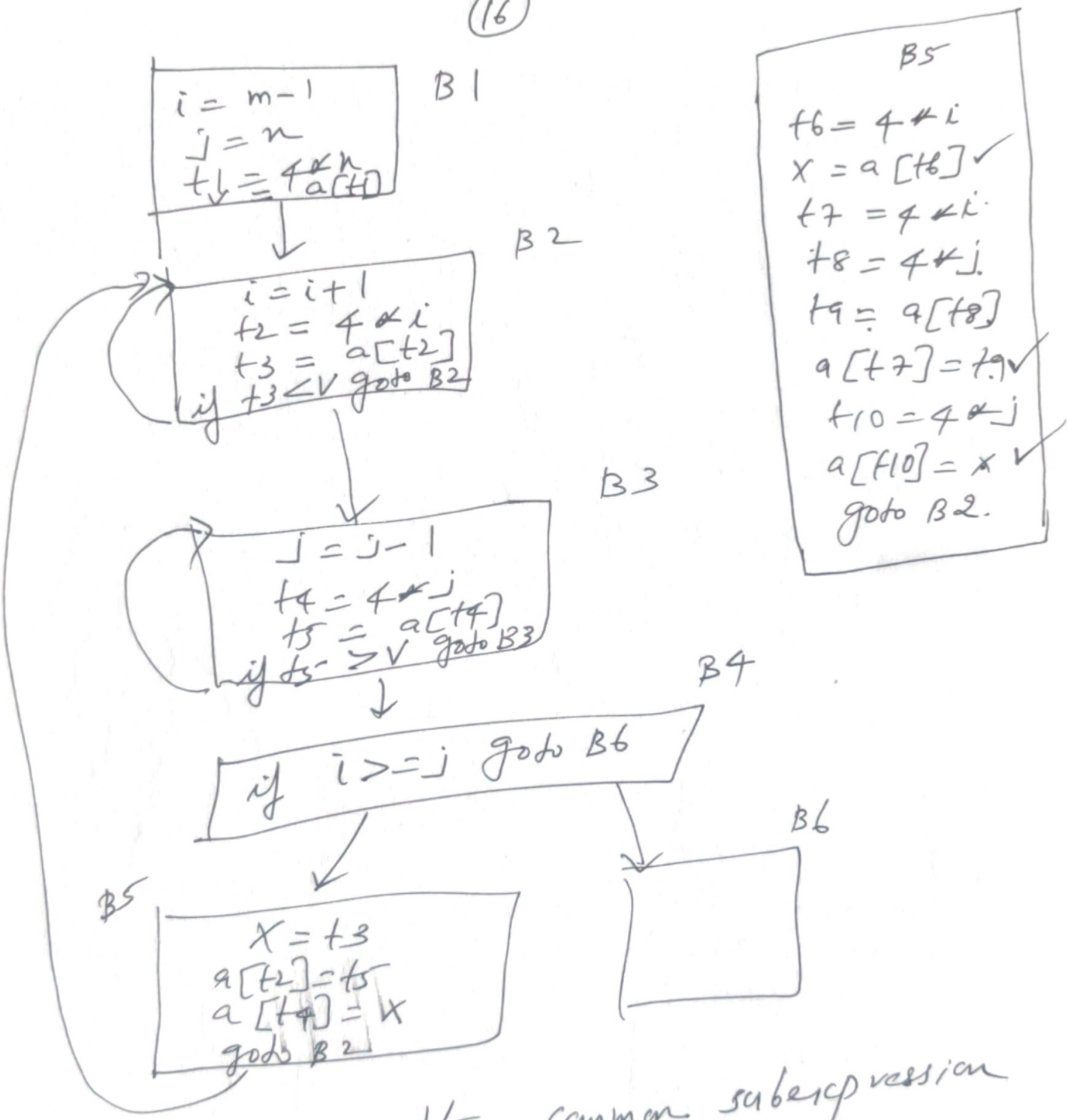


Fig. B5 after common subexpression elimination.

Copy propagation

* Copy propagation is the process of replacing the occurrence of targets of direct assignment with their values. A direct assignment is an instruction of the form $x = y$, eg. $y = x$, $z = 3+y$. Copy propagation would yeild $z = 3+x$.

Dead code elimination

* Dead code elimination removes code that does not affect the pgm. Removing such code has 2 benefits. It shrinks pgm. size, It also reduces running time by avoiding execution of irrelevant operations.
* Dead code includes code that can never be executed or unreachable code.

Example1,
```
debug = false
if (debug) print ....
```

Example2
```
int sum()
{
int a = 24;
int b, c;
c = a << 2;
return c;
b = 24; /* unreachable code */
}
```

Constant folding is the process of simplifying constant expressions during compile time.

$i = 10 + 20;$

Loop optimizations

* Very important place for optimizations, ~~name~~ especially inner loops where programs tend to spend the bulk of their time.
* Three techniques important for loop optimization are - Code motion (which moves the code outside a loop), induction variable elimination (which are apply to eliminate i and j from inner loops) reduction in strength (which replaces an expensive operation by a cheaper one, such as multiplication by an addition)

Code motion

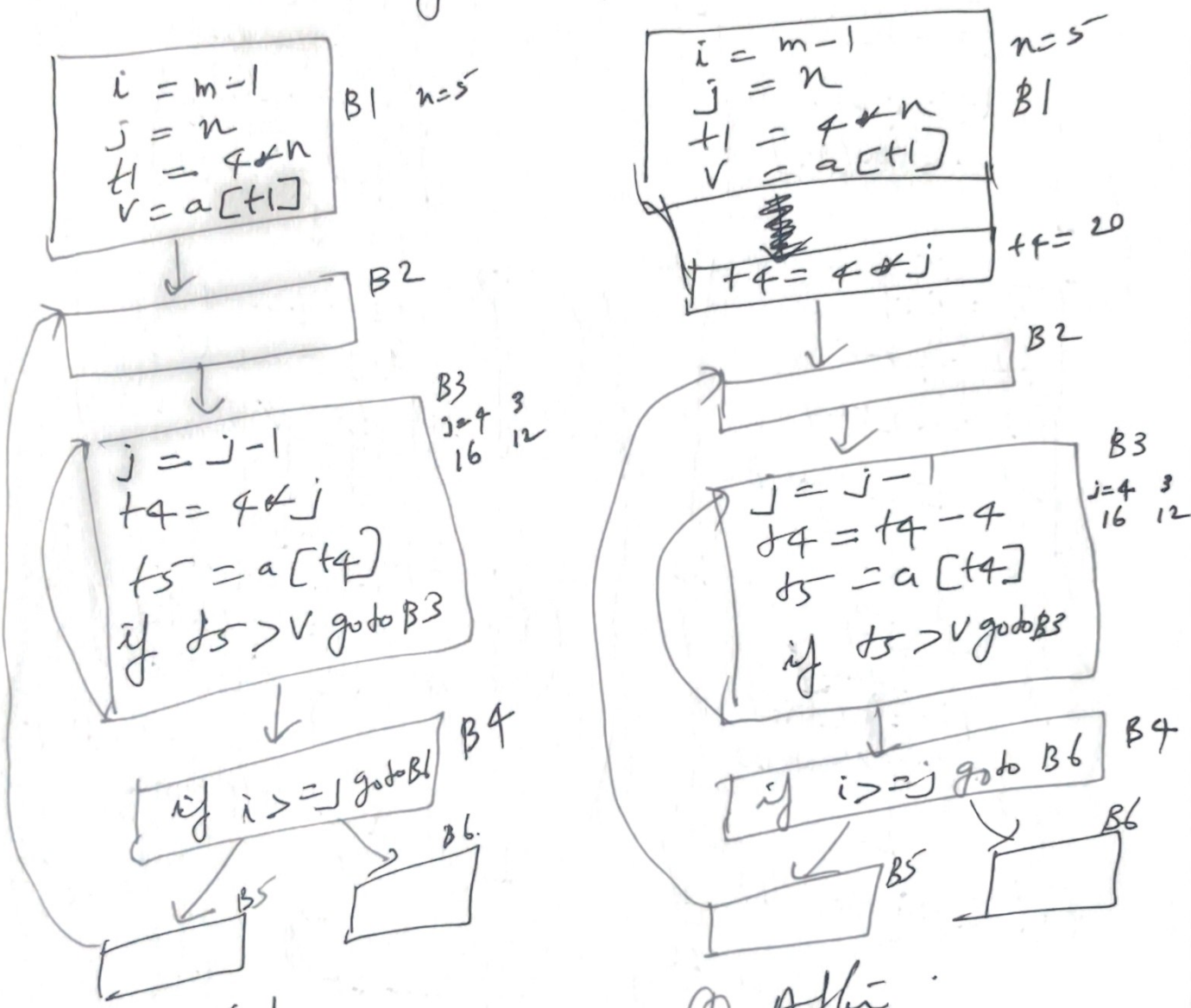
* An important modification that decreases the amount of code in a loop is code motion. For example, evaluation of limit-2 is a loop invariant computation ~~of limit-2~~ in the following while statement.

while (i <= limit-2), Code motion will result in

```
t = limit - 2
while (i <= t)
```

Induction variables and reduction in strength

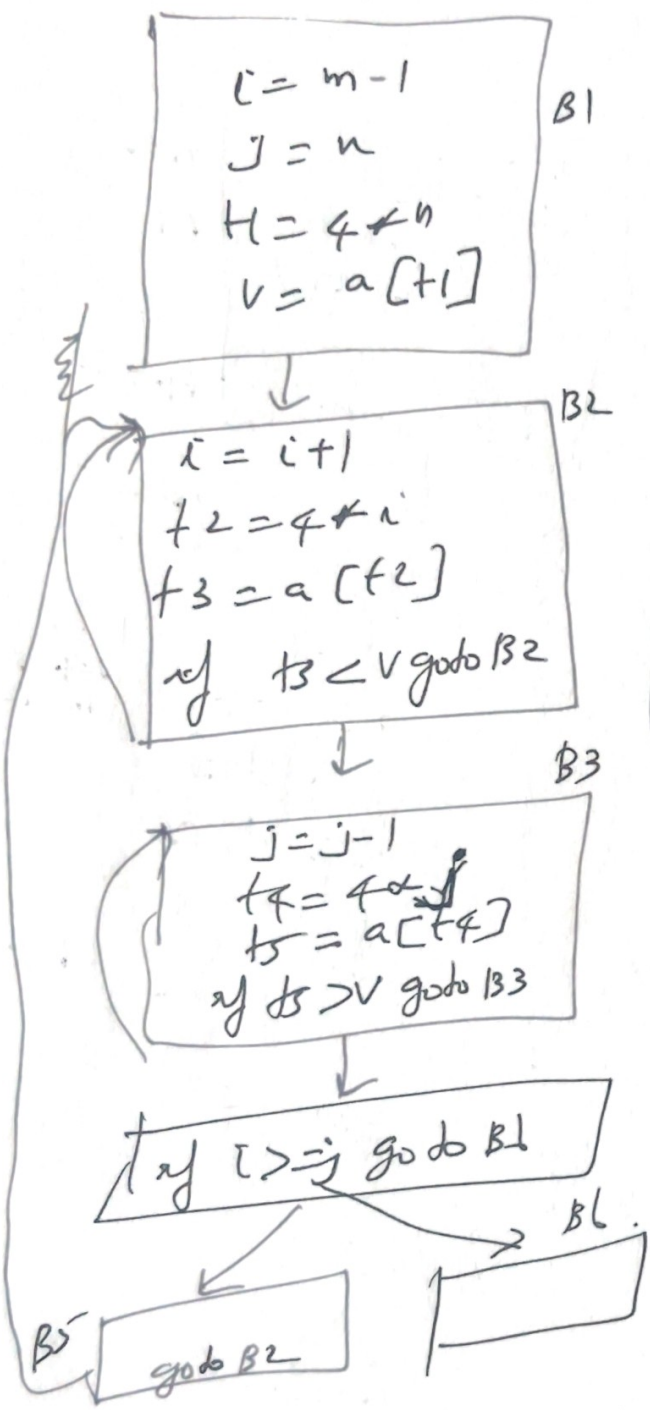
Consider following example,



(a) Before

(b) After

Fig. Strength reduction applied to 4*j in block B2.



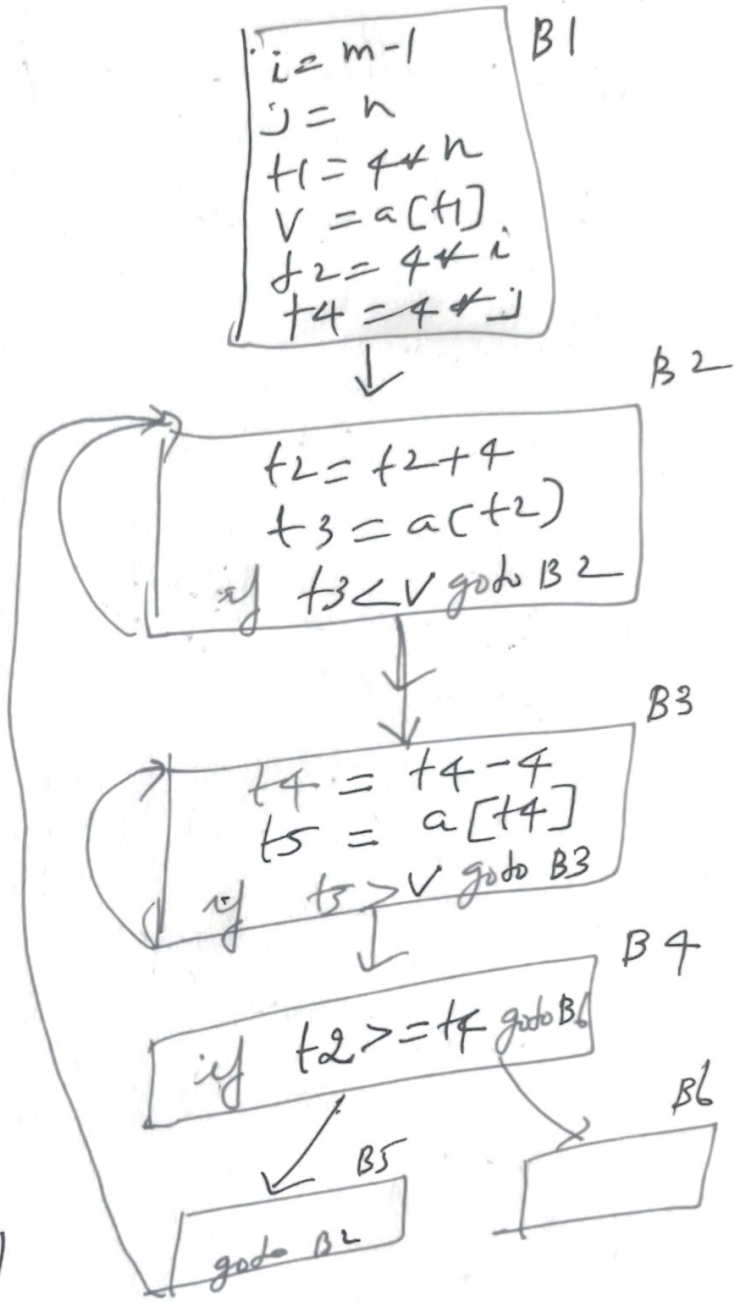


Fig. Flow graph after induction variable elimination.